

After all this, the Linux RTC subsystem can use the registered protocol driver's read and write routines to read and write the RTC.

RTC DS1347 hardware overview

RTC DS1347 is a low current SPI compatible real time clock. The information it provides includes the seconds, minutes and hours of the day, as well as what day, date, month and year it is. This information can either be read from or be written to the RTC DS1347 using the SPI interface. RTC DS1347 acts as a slave SPI device and the microcontroller connected to it acts as the SPI master device. The CS pin of the RTC is asserted 'low' by the microcontroller to initiate the transfer, and de-asserted 'high' to terminate the transfer. The DIN pin of the RTC transfers data from the microcontroller to the RTC and the DOUT pin transfers data from the RTC to the microcontroller. The SCLK pin is used to provide a clock by the microcontroller to synchronise the transfer between the microcontroller and the RTC.

The RTC DS1347 works in the SPI Mode 3. Any transfer between the microcontroller and the RTC requires the microcontroller to first send the command/address byte to the RTC. Data is then transferred out of the DOUT pin if it is a read operation; else, data is sent by the microcontroller to the DIN pin of the RTC if it is a write operation. If the MSB bit of the address is one, then it is a read operation; and if it is zero, then it is a write operation. All the clock information is mapped to SPI addresses as shown in Table 1.

Read address	Write address	RTC register	Range
0x81	0x01	Seconds	0 - 59
0x83	0x03	Minutes	0 - 59
0x85	0x05	Hours	0 - 23
0x87	0x07	Date	1 - 31
0x89	0x09	Month	1 - 12
0x8B	0x0B	Day	1 - 7
0x8D	0x0D	Year	0 - 99
0x8F	0x0F	Control	00H - 81H
0x97	0x17	Status	03H - E7H
0xBF	0x3F	Clock burst	-

Table 1: RTC DS1347 SPI register map

When the *clock burst* command is given to the RTC, the latter will give out the values of seconds, minutes, hours, the date, month, day and year, one by one, and continuously. The *clock burst* command is used in the driver to read the RTC.

The Linux RTC subsystem

The Linux RTC subsystem is the interface through which Linux manages the time of the system. The following procedure is what the driver goes through to register the RTC with the Linux RTC subsystem.

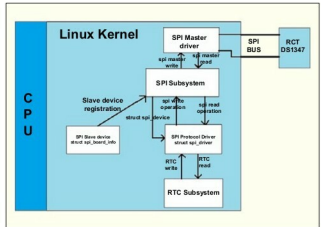


Figure 1: RTC DS1347 driver block diagram

1. Specify the driver's RTC read and write routines through the function pointer interface provided by the RTC subsystem.
2. Register with the RTC subsystem using *devm_rtc_device_register API*.

The RTC subsystem requires that the driver fill the *struct rtc_class_ops* routine, which has the following function pointers.

read_time: This routine is called by the kernel when the user application executes a system call to read the RTC time.

set_time: This routine is called by the kernel when the user application executes a system call to set the RTC time.

There are other function pointers in the structure, but the above two are the minimum an interface requires for an RTC driver.

Whenever the kernel wants to perform any operation on the RTC, it calls the above function pointer, which will call the driver's RTC routines.

After the above RTC operations structure has been filled, it has to be registered with the Linux RTC subsystem. This is done through the kernel API:

```
devm_rtc_device_register(struct device *dev, const char
*name, const struct rtc_class_ops *ops, struct module
*owner);
```

The first parameter is the device object, the second is the name of the RTC driver, the third is the driver RTC operations structure that has been discussed above, and the last is the owner, which is *THIS_MODULE* macro.

Registering the RTC DS1347 as an SPI slave device

The Linux kernel requires a description of all devices connected to it. Each subsystem in the Linux driver model has a way of describing the devices related to that subsystem. Similarly, the SPI subsystem represents devices based on the SPI bus as a *struct spi_device*. This structure defines the SPI slave device connected to the processor running the Linux kernel. The device structure is written in the board file in the